



揭示Java泛型编程技术背后的“内幕”

Java泛型“擦除”原理

北京理工大学计算机学院
金旭亮

泛型类型的“擦除”原理

定义：

```
public class Pair<T> {  
    private T first;  
    private T second;  
}
```

使用：

```
Pair<MyClass> buddies = ... ;  
MyClass buddy = buddies.getFirst();
```



Java编译器会将Pair<T>中的T转为Object，以便在JVM上运行，这就是“**擦除 (erasure)**”。



Java编译器为什么要这样干？其原因是——
Java虚拟机不直接支持泛型！

泛型方法参数“擦除”实例 (1)

```
public static <T extends Comparable> T min(T[] a)
```

擦除之后



```
public static Comparable Min(Comparable[] a)
```

泛型方法参数“擦除”实例（2）

如果泛型方法中的泛型参数拥有多个约束：

```
<T extends Comparable & Serializable> T mtd(){ }
```



按第一个约束进行“擦除”

```
Comparable mtd() { }
```



如果泛型方法中的泛型参数没有约束，则使用Object替换。

instanceof 运算符

- instanceof 运算符应用于泛型时，只处理“擦除”后的原始类型

```
if (a instanceof Pair<String>)  
...
```



```
if(a instanceof Pair)  
...
```



到底有几个泛型类？

```
List<String> strList = new ArrayList<String>();  
List<Integer> intList = new ArrayList<Integer>();  
  
System.out.println(  
    strList.getClass() == intList.getClass()  
);
```

因为“擦除”，上述代码输出：**true**



在程序运行时，所有泛型参数“都消失了”，我们只能得到一个普通类型（不再是泛型类型），对应唯一的一个Class对象。

知识应用检测

- 以下代码无法编译，为什么？

```
public class MyClass<T> {  
    public boolean equals(T value) {  
        return true;  
    }  
}
```



参考答案



前页的示例代码，Java编译器在擦除泛型参数之后，将出现两个“一模一样”的equals方法，一个是MyClass<T>所定义的，另一个则是Object所定义的，这将造成混乱。因此，Java编译器报告一个编译错误而中止编译过程。

学习建议



- Java泛型编程技术比较复杂，你必须知道JVM是如何处理泛型的，才能正确地使用泛型。
- 牢记Java编译器会“擦除”所有泛型参数，有助于摸清Java泛型的“怪脾气”。
- 在学习过程中，一定要编写测试代码，再找一些相关的技术书籍和资料精读，才能逐步把握Java泛型编程技术。
- 由于JDK中大量应用了泛型技术，就算你不想写泛型代码，出于用好JDK中现有组件的目的，你也必须掌握Java泛型编程的基础知识和基本技能，这一关是必须要过的。

小结



本讲介绍了Java泛型的“擦除”原理，它是javac编译器处理Java泛型的基本技术手段。之所以Java要采取这种“别扭”的方式实现泛型，是由于JVM在最初设计时就没考虑要支持泛型，后来想加入，但又必须考虑到兼容性，所以不得已而采用这种“打补丁”的方法。



尽管Java泛型是“伪泛型”，但我们还是应该在开发中使用它，因为使用它，编译器就可以检测出很多有问题的代码，从而有助于提升代码的可维护性。